Arquitectura del sistema — Proyecto MiBici

1 · FUENTE DE DATOS

Portal de datos abiertos de MiBici

CSVs mensuales de viajes (ene–dic 2025, 4.5M registros) + nomenclatura de estaciones (484, con lat/lon y capacidad).

POR QUÉ: Fuente oficial y pública. Los nombres de columna y la codificación varían entre archivos: tratarlos es parte del pipeline, no un paso manual.

▼

2 · INGESTA

mibici/dataset.py — Python (requests + pandas)

Descarga idempotente · normaliza columnas, encoding (utf-8/latin-1) y fechas · un solo comando, sin pasos manuales.

POR QUÉ: La ingesta manual es deuda técnica (pipeline jungles, Sculley et al., 2015); un script ejecutable garantiza que cualquier integrante reconstruye los mismos datos.

▼

3 · ALMACENAMIENTO LOCAL

data/raw (CSV) → data/processed (Parquet)

Crudos intactos para auditoría; procesados en formato columnar.

POR QUÉ: Parquet es columnar y comprimido: leer 4.5M de filas tarda segundos en vez de minutos. Los datos nunca entran a Git.

▼

4 · VERSIONADO

manifest.json (SHA-256 por archivo → data_version) + Git/GitHub

Código en Git con rama protegida y Pull Requests revisados · datos versionados por manifiesto de hashes.

POR QUÉ: El modelo depende de código Y datos; el manifiesto da trazabilidad sin infraestructura extra — data_version 7aa5e436d887 identifica exactamente este corte.

▼

5 · FEATURES Y ENTRENAMIENTO

mibici/features.py + mibici/modeling/ — scikit-learn

Funciones de features reutilizables y documentadas · P1 regresión · P2 clasificación · P3 clustering · config.py centraliza rutas, fechas y semilla (SEED=42) · partición temporal sin fuga del futuro.

POR QUÉ: La configuración dispersa es deuda de configuración (Sculley et al., 2015); la lógica vive en el módulo, no en notebooks — los notebooks son solo exploración.

▼

6 · TRACKING DE EXPERIMENTOS

MLflow Tracking (backend SQLite local)

Experimento único mibici-equipo32 · ≥1 run por pregunta · params (hiperparámetros, fechas, data_version) · métricas nombradas por conjunto (test_mae) · tags (pregunta, git_commit, data_version) · artefactos (matriz de confusión, mapa de clusters).

POR QUÉ: Sin tracking los experimentos no son comparables ni reproducibles; los tags git_commit + data_version amarran cada resultado al código y a los datos exactos que lo produjeron.